



Web Fundamentals

WEF1VO

Teil 11: JavaScript Workflow & Tools

Medientechnik und -design | WS 2025/26

Einführung

Warum überhaupt Tools und welche?



Überall Neues

- JavaScript: Node.js, Frontend-Frameworks
- CSS: Neue Spezifikationen
- Performance & Codequalität: wichtig

Ziel: Bessere Projekte!



Aber auch Limits

- CSS hat seine Einschränkungen
- Viel händische Arbeit nötig:
 - Codequalität?
 - Reload bei Änderung (F5)?
 - Files klein halten?

```
p {  
  margin-top: 10px;  
  margin-bottom: 10px;  
  color: red;  
}  
  
div {  
  margin-top: 10px;  
  margin-bottom: 10px;  
  color: black;  
}  
  
section {  
  margin-top: 10px;  
  margin-bottom: 10px;  
}
```

Tools & Workflow

S

To the Rescue!





Tools = Binaries



Bash-Scripts, Ruby-Gems, PowerShell-Scripts uvm.

Node.js

JavaScript Runtime



Node.js Webserver



In einer Datei `nodeserver.mjs`: ← ECMAScript Module (ESM)

```
import {createServer} from "node:http";

const hostname = "127.0.0.1";
const port = 3000;

const server = createServer((request, response) => {
  response.statusCode = 200;
  response.setHeader("Content-Type", "text/plain");
  response.end("Hello World!");
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Install: nodejs.org

winget install OpenJS.NodeJS

brew install node

Download Node.js®

Get Node.js® v25.4.0 (Current) ▾ for Windows ▾ using Docker ▾ with npm ▾

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:25-alpine
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:25-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v25.4.0".
12
13 # Verify npm version:
14 npm -v # Should print "11.7.0".
```

PowerShell

[</> Copy to clipboard](#)

Docker is a containerization platform. If you encounter any issues please visit [Docker's website](#) ↗

Or get a prebuilt Node.js® for Windows ▾ running a x64 ▾ architecture.

[Windows Installer \(.msi\)](#)

[Standalone Binary \(.zip\)](#)

Read the [changelog](#) ↗ or [blog post](#) for this version.

Learn more about [Node.js releases](#), including the release schedule and LTS status.

Learn how to [verify](#) ↗ signed SHASUMS.

Looking for Node.js source? Download a signed [Node.js source](#) ↗ tarball.

Check out our [nightly](#) ↗ binaries or all [previous releases](#) or the [unofficial](#) ↗ binaries for other platforms.

Proudly supported by the partners below:

We are able to serve Node.js's downloads and maintain our infrastructure proudly due to the support of these partners, and more.



Node.js Commandline



Installation überprüfen:

```
> node -v
```

```
v25.4.0
```

Webserver-Beispiel:

```
> node nodestserver.mjs
```

Node.js & npm

Packages/Module verwenden



npm Commandline



Installation überprüfen:

```
> npm -v
```

```
11.8.0
```

Packages global installieren:

```
> npm install -g <package>
```

Packages lokal installieren:

```
> npm install <package>
```

Lokale Packages verwalten:

```
> npm init
```

Alles klar?

Let's go!



Sass

Syntactically Awesome Stylesheets
Eine Stylesheet-Sprache und CSS-Präprozessor

Funktionsweise



sass



<https://www.npmjs.com/package/sass>

Package global installieren:

```
> npm install -g sass
```

Präprozessor ausführen:

```
> sass input.scss output.css
```

Sass: Variablen



SCSS:

```
$main-color: #bada55;  
$main-font: system-ui, sans-serif;  
$bg-color: rgb(41 49 51);
```

```
body {  
  font-family: $main-font;  
  color: $main-color;  
  background-color: $bg-color;  
}  
  
a:link, a:visited,  
a:hover, a:active {  
  color: $main-color;  
}
```

CSS:

```
body {  
  font-family: system-ui, sans-serif;  
  color: #bada55;  
  background-color: rgb(41, 49, 51);  
}  
  
a:link, a:visited, a:hover, a:active {  
  color: #bada55;  
}
```

Sass: Verschachtelung



SCSS:

```
nav {  
  ul {  
    list-style: none;  
    padding: 0;  
  }  
  li {  
    display: inline-block;  
  }  
  a {  
    text-decoration: none;  
  }  
}
```

CSS:

```
nav ul {  
  list-style: none;  
  padding: 0;  
}  
nav li {  
  display: inline-block;  
}  
nav a {  
  text-decoration: none;  
}
```

Sass: Partial & Module



SCSS – _variables.scss:

```
$main-color: #bada55;  
$main-font: system-ui, sans-serif;  
$bg-color: rgb(41 49 51);
```

SCSS – main.scss:

```
@use "variables";
```

```
body {  
  font-family: variables.$main-font;  
  color: variables.$main-color;  
  background-color: variables.$bg-color;  
}
```

CSS:

```
body {  
  font-family: system-ui, sans-serif;  
  color: #bada55;  
  background-color: rgb(41, 49, 51);  
}
```

Sass: Mixins



SCSS:

```
@mixin padding-margin($pad, $mar) {  
  padding-left: $pad;  
  padding-right: $pad;  
  margin-top: $mar;  
  margin-bottom: $mar;  
}
```

```
p {  
  @include padding-margin(10px, 20px);  
}
```

CSS:

```
p {  
  padding-left: 10px;  
  padding-right: 10px;  
  margin-top: 20px;  
  margin-bottom: 20px;  
}
```

Sass: Vererbung



SCSS:

```
.message {  
  border: 3px solid #ccc;  
  padding: 10px;  
  color: #333;  
}  
  
.status {  
  @extend .message;  
  border-color: green;  
}  
  
.error {  
  @extend .message;  
  border-color: red;  
}
```

CSS:

```
.message, .status, .error {  
  border: 3px solid #ccc;  
  padding: 10px;  
  color: #333;  
}  
  
status {  
  border-color: green;  
}  
  
.error {  
  border-color: red;  
}
```

Aber halt ...

Bei jeder Änderung in der SCSS-Datei sass
aufrufen!?

```
sass _scss/main.scss css/main.css
```

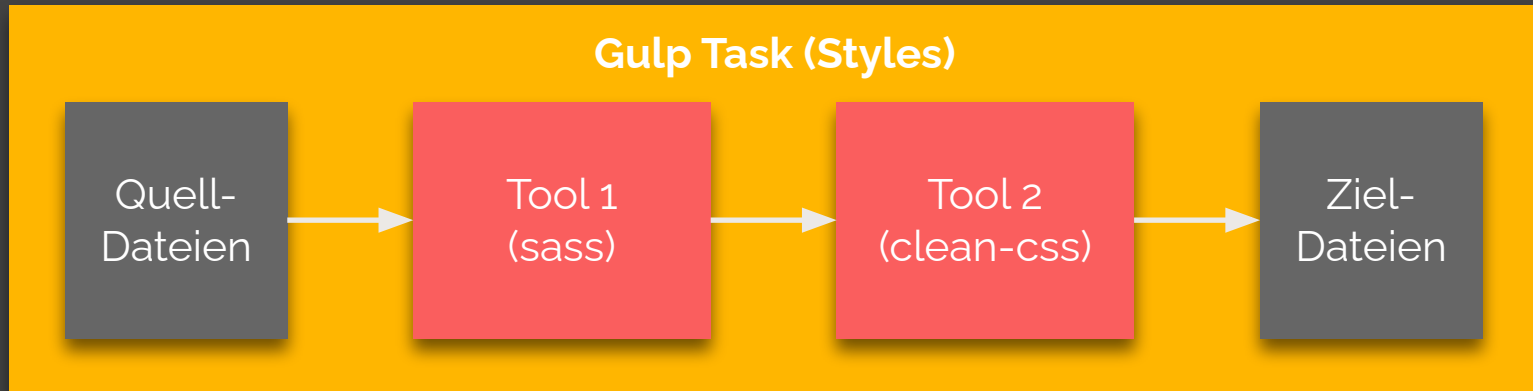


3

gulp

Task-Runner & Build Tool

Funktionsweise



gulp Commandline



Gulp CLI global installieren:

```
> npm install -g gulp-cli
```

package.json im Projekt anlegen:

```
> npm init --yes
```

Gulp lokal installieren:

```
> npm install gulp
```

+ gulpfile.mjs erzeugen

gulp Tasks



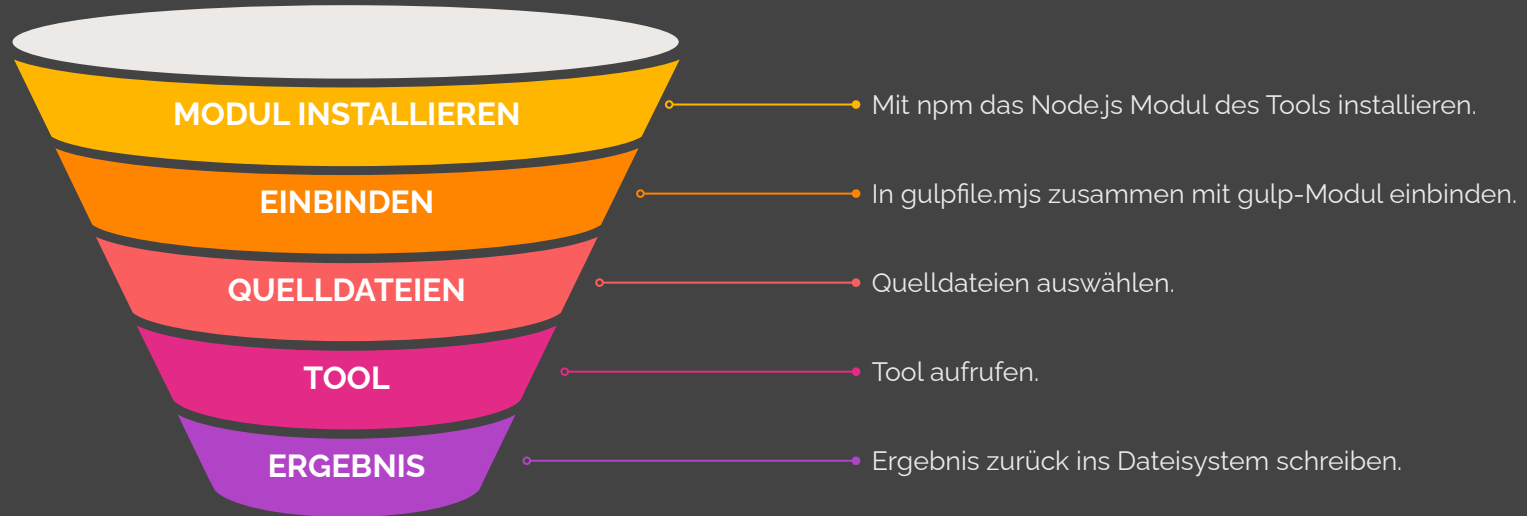
Definiert in gulpfile.mjs

```
export function test(cb) {  
  console.log("Hallo Welt!");  
  cb();  
}
```

Ausführen:

```
> gulp test
```

Komplexere Tasks



JavaScript Minification



Node.js Modul installieren:

```
> npm install -D gulp-terser
```

1

Task definieren:

```
import { src, dest } from "gulp";  
import terser from "gulp-terser";
```

2

```
exports function scripts()  
  return src("_js/**/*.js")
```

3

```
  .pipe(terser())  
  .pipe(dest("js"));
```

4

5

```
}
```

Ausführen:

```
> gulp scripts
```

Style Minify &

Sourcemaps

Node.js Module installieren:

```
> npm install -D sass gulp-sass gulp-clean-css gulp-sourcemaps
```

Task definieren:

```
import * as dartSass from "sass";
import gulpSass from "gulp-sass";
import cleanCSS from "gulp-clean-css";
import sourcemaps from "gulp-sourcemaps";

const sass = gulpSass(dartSass);

export function styles() {
  return src("_scss/main.scss")
    .pipe(sourcemaps.init())
    .pipe(sass())
    .pipe(cleanCSS())
    .pipe(sourcemaps.write())
    .pipe(dest("css"));
}
```

Ausführen:

```
> gulp styles
```



JavaScript Linting



Node.js Modul installieren:

```
> npm install -D gulp-eslint-new
```

Task definieren:

```
import esLintNew from "gulp-eslint-new";
```

```
export function lint() {  
  return src("_js/**/*.js")  
    .pipe(esLintNew())  
    .pipe(esLintNew.format())  
    .pipe(esLintNew.failAfterError())  
}
```

eslint.config.mjs:

```
export default [  
  {  
    rules: {  
      "semi": "error",  
      "indent": "warn",  
      "prefer-const": "error"  
    }  
  }  
];
```

Ausführen:

```
> gulp lint
```

Testserver



Node.js Modul installieren:

```
> npm install -D browser-sync
```

Task definieren:

```
import browserSync from "browser-sync";
```

```
const bs = browserSync.create();
```

```
export function server(done) {
```

```
  bs.init({  
    server: {  
      baseDir: "./"  
    }  
  });
```

```
  done();  
}
```

Ausführen:

```
> gulp server
```

Testserver: **Watcher**



Task definieren:

```
import { src, dest, watch, series } from "gulp";
```

```
function reload(done) {  
  bs.reload();  
  done();  
}
```

```
export function watcher() {  
  watch("_scss/**/*.scss", series(styles, reload));  
  watch("_js/**/*.js", series(scripts, reload));  
  watch("./*.html", reload);  
}
```

Ausführen:

```
> gulp watcher
```

Default-Task mit allem



Task definieren:

```
import { src, dest, watch, series, parallel } from "gulp";  
  
export default series(parallel(styles, scripts), server, watcher);
```

Ausführen:

```
> gulp
```

Klingt gut?

Ausprobieren (in der Übung)!





Nächstes Mal in WEF1VO

HTML JavaScript APIs

Credits



- S. 7: Node.js Logo: <https://nodejs.org/en/about/branding>
- S. 11: NPM Logo: <https://github.com/npm/logos>
- S. 15: Sass Logo: <https://sass-lang.com/styleguide/brand>